

****Title:****

Reversible Rank-Based Storage and Retrieval Using Entropy-Aligned Encoding

****Author:****

Trent Wyatt

Abstract

This paper presents a novel, deterministic approach to encoding data as numeric offsets using rank/unrank transformations derived from entropy-aligned symbol sets. We propose a reversible storage and retrieval model in which every datum is mapped to a unique location using bijective, content-aware indexing. This technique balances entropy efficiency, access speed, and practical storage by unifying adaptive compression models with numeric address spaces. The method is demonstrated through formal reasoning and practical implementation, offering a framework for scalable, collision-free, content-addressable systems. We additionally demonstrate how this model can be applied to lightning-fast public key infrastructure (PKI) lookup and identity verification.

1. Introduction

Traditional storage systems separate content from address. Hashing and file systems use opaque pointers or hash digests to access data, often introducing indirection, collisions, or entropy waste. This paper proposes a tightly coupled alternative: an address **is** the data - or more precisely, its **entropy-optimal rank** in a structured symbol space.

The core system involves four primitives:

1. ``analyze(data)``: Evaluate symbol distributions and determine the optimal compression or ranking model.
2. ``rank(data)``: Map the data to a unique, entropy-optimal integer offset.
3. ``unrank(offset)``: Reconstruct the original data from the numeric address.
4. ``store(offset)``: Use the numeric offset directly as a persistent or in-memory location.

We show that this system provides an ideal balance of compression ratio, deterministic access, and computational simplicity.

2. Entropy and Efficiency

2.1 Rank as Minimal Encoding

Each fixed-length string over a known alphabet can be interpreted as a number in base-N, where N is the alphabet size. For an alphabet of size C, and a string of length L, the total number of unique strings is C^L . This gives us a perfect numeric space to encode and retrieve any string bijectively.

The entropy per character is $H = \log_2(C)$, and each string contains $L * H$ bits of entropy. By encoding the string as an integer in base C, we use exactly that entropy with no redundancy.

2.2 Symbol Model Optimization

If the symbol distribution is known or analyzable, a more optimal compression model can be chosen - Huffman coding, ANS, or even arithmetic coding. Each of these can be used to assign a rank to a message in a dense entropy space. Crucially, all are reversible and maintain bijection with numeric offsets.

3. Deterministic Storage and Retrieval

3.1 The Rank-Unrank API

Given that data can be uniquely ranked, we use the numeric rank as the key into a deterministic storage map. The reverse operation (unrank) reconstructs the data from this key without any additional metadata.

```
rank(data)      // data -> offset
unrank(offset)  // offset -> data
```

These two functions define a complete, bijective system.

3.2 Optimizing Storage

By storing blocks or pages at their offset or within an offset-derived file system, access is immediate:

```
offset = rank(data);
store_at(offset, data);
```

Retrieval is deterministic:

```
data = unrank(offset);
```

3.3 Adaptive Models

Using ``analyze(data)``, we can select from multiple symbol models (e.g., tokenized text, JPEG DCT bands, etc.) and dynamically switch the ranking basis. This enables domain-optimized entropy compression and layout.

4. Practical Considerations

4.1 Complexity

- Rank/unrank operations are $O(L)$ for fixed base- N alphabets.
- Huffman and ANS models maintain $O(1)$ decode time with small lookup tables.
- No collisions, no table probes, no hashing - just math.

4.2 Address Space Size

- For strings of length 8 over a base-64 alphabet: $64^8 = 2^{48}$ unique entries.
- Easily mapped to 64-bit offsets.

4.3 Applications

- Compressed file systems
- Blockchain traversal (rank = block number)
- Zero-knowledge storage proofs
- PKI and digital identity systems

In the context of PKI, our system provides a collision-free and deterministic mapping from any user identity string (e.g., certificate serial number, public key fingerprint, or DNS name) directly to a storage offset. This allows lightning-fast lookup of certificates or verification records using only a reversible, bijective function - eliminating the need for hash tables, collision resolution, or external indexing.

Given a public key fingerprint such as "219874650102398456", our system directly maps this to a numeric offset using `rank()` over a numeric alphabet. Storage is keyed at that offset. When verifying identity, the fingerprint is ranked again and the record is retrieved with zero ambiguity.

5. Conclusion

This work demonstrates a mathematically minimal, entropy-aligned, and practically deployable storage model. Rank-based addressing removes the need for traditional identifiers, replaces hashes with bijective arithmetic, and simplifies traversal into pure function calls. We have further shown that this method is ideally suited for cryptographic identity systems like PKI, where fast, deterministic, and collision-free key mapping is paramount.

Future Work:

- Parallel rank/unrank pipelines
- Hardware acceleration for symbol models
- Self-verifying blockchains with rank-referenced proofs
- Certificate transparency maps indexed by entropy-aligned ranks

References

- Cover & Thomas, Elements of Information Theory
- Duda et al., Asymmetric Numeral Systems
- Knuth, The Art of Computer Programming, Vol. 4
- Huffman, A Method for the Construction of Minimum-Redundancy Codes